



10

14

1680 sp

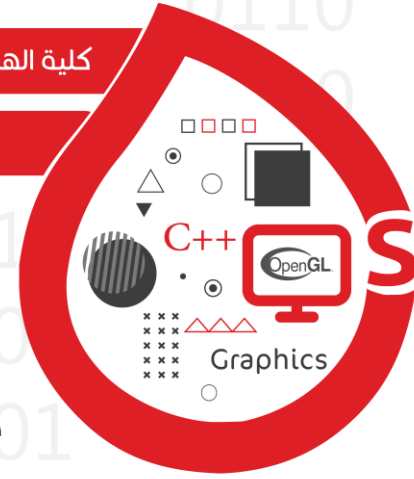
نظري

كلية الهندسة المعلوماتية

السنة الثالثة

Filling area

د. عمار النحاس



RB Informatcs; 09/12/2025

بيانيات حاسوبية

السلام عليكم و رحمة الله و بركاته

تحدثنا في المحاضرة السابقة عن عملية ال scan conversion وقلنا انها تتم عن طريق خوارزمية ال Rasterization التي تحول الغرض من التمثيل الهندسي إلى Pixels على الشاشة حيث يكون دخل الخوارزمية هي التمثيل الهندسي والخرج هو النقاط التي سيتم إضاءتها.

تحدثنا عن خوارزمية رسم المستقيم والدائرة , لكن ماذا عن المضلعات؟

نحتاج إلى إضاءة عدد كبير من ال pixels لرسم مضلع مملوء, أي إضاءة البكسلات الداخلية.

ولدينا مشكلتان أساسيتان:

- ماهي البكسلات التي علينا إضاءتها.
- مالون أو قيمة كل (R,G,B) pixels وسنعالج ذلك في المحاضرة القادمة.

سننتحدث أولاً عن حل مشكلة filling area

سنفكر بحل بسيط, وهو المرور على جميع بكسلات الشاشة وتعويض كل نقطة منها في معادلة الشكل وبالتالي تحديد مكان البكسل بالنسبة لهذا الشكل ثم إضاءته أو إطفائه حسب مكانه. هذا الحل يعتبر سيء جداً لأنه يحتاج وقتاً طويلاً في التنفيذ وهذا حل Brute Force .

■ أنواع ال polygon :

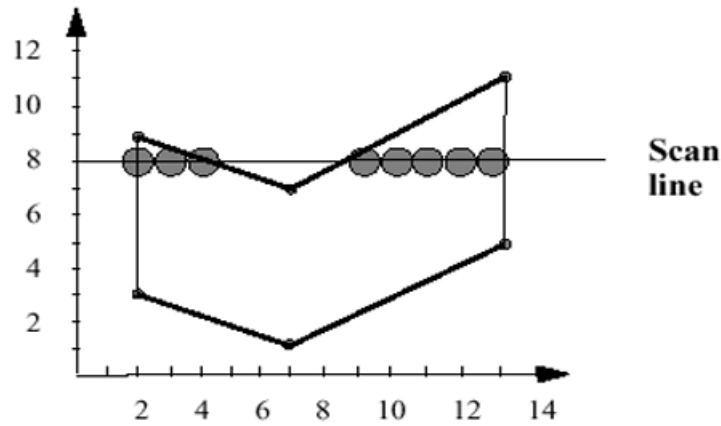
1. محدب (Converse) : جميع زواياه أصغر من 180 درجة.
2. مقعر (Concave) : زاوية وحدة أو أكثر أكبر من 180 درجة.
3. بسيط (Simple) : لا تتقاطع أضلاعه الا عند الرؤوس مثل المربع.
4. معقدة (Complex) : قد تتقاطع أضلاعه مثل المضلعات التي تتقاطع مع نفسها كالنجمة الخماسية.

ملاحظة ارييسيزية:

OpenGL لا تدعم المضلعات المقعرة ولا المضلعات المعقدة (مركبة) إذاً نحن بحاجة لخوارزمية عامة تستطيع تعبئة أي شكل مهما كان معقداً.

Scan line algorithm

فكرة الخوارزمية: تقوم بمسح المضلع بخط مستقيم يوازي محور x ويحرك بمقدار pixel واحد من أخفض رأس في المضلع إلى أعلى رأس فيه أي من أخفض y إلى أعلى y و في كل حد نحو الأعلى نقوم بإضاءة النقاط من المستقيم التي تقع داخل المضلع.



ملاحظة:

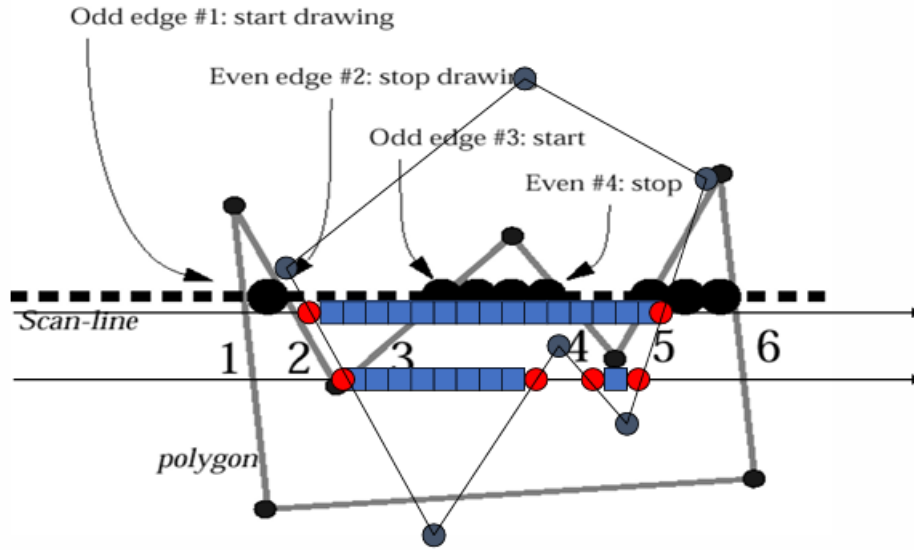
- إذا كان المضلع مقعر أو محدب يمكن أن يكون التقاطع بين المستقيم الأفقي والمضلع بأكثر من نقطتين.

- إذا كان المضلع محدب أو بسيط يكون التقاطع إما بنقطتين فقط أو ولا نقطة.

ولكن كيف نحدد ماهي النقاط التي تقع داخل الشبكة او خارجها أثناء تطبيق خوارزمية Scan line ؟

لو كان للمضلع معادلة لكان من السهل التعويض فيها لكننا بحاجة لحل اخر :

- في كل حركة نحو الأعلى سنقوم بالمسح من اليسار إلى اليمين.
- نحسب عدد مرات التقاطع بين القطعة المستقيمة و الأضلاع حيث يبدأ العد من اليسار إلى اليمين وبدءاً من العدد 1.
- نتبع قاعدة odd-parity :
- عند الوصول إلى نقطة تقاطع عددها فردي نبدأ بإضاءة البكسلات.
- نستمر بإضاءة البكسلات حتى الوصول إلى نقطة التقاطع عددها زوجي.
- عند النقطة التي عددها زوجي نتوقف عن رسم البكسلات حتى الوصول إلى النقطة الفردية التالية وهكذا.....



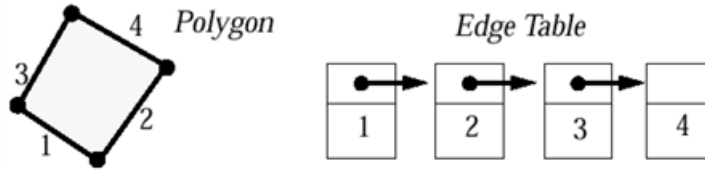
عند كل مستوى سنقوم بإيجاد نقاط التقاطع كما ذكرنا ولإيجاد نقاط التقاطع لابد لنا أولاً من معرفة ماهي الأضلاع التي يتقاطع خط ال **Scan line** معها لذلك سنقوم بإنشاء مجموعتين :

Edge Table(ET)	Active Edge Tabel (AET)
----------------	-------------------------

كلاهما linked list

1. Edge Table :

- تتضمن جميع أضلاع الشكل التي لم تتعالج بعد.
- نعلم أنه لكل ضلع من الأضلاع بداية ونهاية على المحور y لذا تكون الأضلاع في هذه المجموعة مرتبة تصاعدياً حسب قيمة y للرأس السفلي للضلع .



2. Active Edge Table :

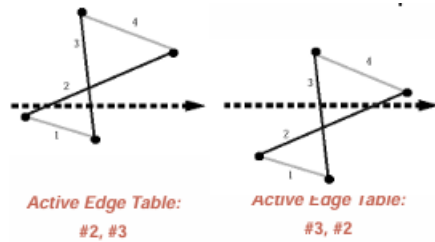
- تتضمن الأضلاع التي تتقاطع مع ال scan line الحالي وتسمى بالأضلاع الفعالة .
- تكون الأضلاع مرتبة تصاعدياً حسب قيمة x لنقطة تقاطع الضلع مع ال scan line ونخزن من أجل كل ضلع نقطة بدايته ونهايته ومقلوب الميل واحد على m (سنرى لم نحتاج مقلوب الميل لاحقاً) و x اخر نقطة تقاطع , ولا نخزن y لأنها ذاتها y ال scan line , ومنها نستنتج إحداثيات نقطة التقاطع الحالية ونخزنها بدلاً من نقطة التقاطع السابقة.

ملاحظات:

1. في نقطة بداية حركة ال scan line سيكون متقاطع مع ضلعين (على الأقل)، لذلك سننقلهم إلى AET ونحذفهم من ET .
2. عندما يصل ال scan line إلى نقطة نهاية أحد الأضلاع نحذف هذا الضلع من AET , وعندما يصل إلى بداية أحد الأضلاع , نضيف الضلع إلى AET ونحذفه من ET .
3. لذلك عند الوصول إلى نقطة نهاية حركة ال scan line ستكون AET , ET فارغتين بالتأكيد.
4. Open GL لا ترسم سوى مثلثات أي تستطيع تحويل أي مضلع إلى مثلثات لذا تقوم بتطبيق خوارزمية scan line بشكل أسهل وأسرع.

ملاحظة هامة:

عندما يكون لدينا مضلع معقد أي المضلعات التي تتقاطع مع نفسها قد يتغير ترتيب الأضلاع نتيجة تقاطعها ، لذا نحن بحاجة لترتيب نقاط التقاطع تبعاً لقيم x وفي كل مرة يتحرك فيها ال scan line به مقدار 1 للأعلى فمثلاً في الصورة اول مرة كان التقاطع مع الضلع 2 قبل 3 و في المرة الثانية كان التقاطع مع 3 قبل 2



Note: Active Edge Table still contains same edges, but x -intercept crossings have rearranged order of edges after sorting

ملاحظة : ماذا يخزن ال AET ؟

يخزن معلومات ال Edge و التي هي اربع معلومات x -intercept, inverse slope, endpoint1, endpoint2 لكل edge

Endpoint1: هي بداية ال edge ما يهمني هوي فقط y و ليس x

Endpoint2: هي النهاية

و يعتبر بداية كل ضلع هي ال y الأقل و نهايته هي ال y الأعلى

Inverse slope : هو عكس الميلان $1/m$ اي فرق الاكسات على فرق الوايات

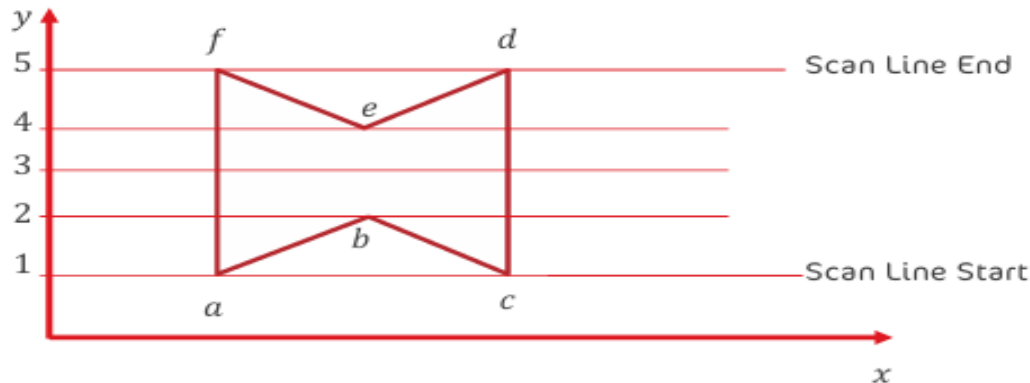
x -intercept : هي ال x لآخر نقطة تقاطع (إحداثيات x لنقطة التقاطع الحالية مع خط المسح (scan line) (الحالي)

Nothing Will Get Easier Until You Become Stronger



تمرين اريسيزي:

مثال:



الحالة الابتدائية:

$$ET = \{ab, af, cb, cd, ef, ed\}, \quad AET = \{\}$$

- عندما يكون الـ Scan Line في البداية أي عند $y = 1$ ، سنجد أنها تقابل رأسين هما a, c لذلك نضيف إلى AET جميع الأضلاع التي بدايتهم a أو c ونحذف هذه الأضلاع من ET، في حالتنا هنالك أربعة أضلاع:
 $\{ab, af, cb, cd\}$

فتصبح المجموعتان بالشكل:

$$ET = \{ef, ed\}, \quad AET = \{ab, af, cb, cd\}$$

- يستمر الـ Scan Line بالارتفاع حتى يصل إلى $y = 2$ ، عندئذ يصل إلى النقطة b التي تمثل نقطة النهاية للأضلاع $\{ab, cb\}$ فنحذف هذين الضلعين من AET

فتصبح المجموعتان بالشكل:

$$ET = \{ef, ed\}, \quad AET = \{af, cd\}$$

- يستمر الـ Scan Line بالارتفاع حتى يصل إلى $y = 3$ ، ولا يتقاطع عندها مع أي نقطة بداية أو نهاية لضلع

فتبقى المجموعتان بالشكل:

$$ET = \{ef, ed\}, \quad AET = \{af, cd\}$$

- يستمر الـ Scan Line بالارتفاع حتى يصل إلى $y = 4$ ، عندئذ يصل إلى النقطة e التي تمثل نقطة البداية للأضلاع $\{ef, ed\}$ فنحذف هذين الضلعين من ET ونضيفهم إلى AET

فتصبح المجموعتين بالشكل:

$$ET = \{\}, \quad AET = \{af, cd, ef, ed\}$$

- يستمر الـ Scan Line بالارتفاع حتى يصل إلى $y = 5$ ، عندئذ يصل إلى النقطتين d, f اللتان تمثلان نقطة النهاية للأضلاع $\{af, cd, ef, ed\}$ فنحذف هذه الأضلاع من AET

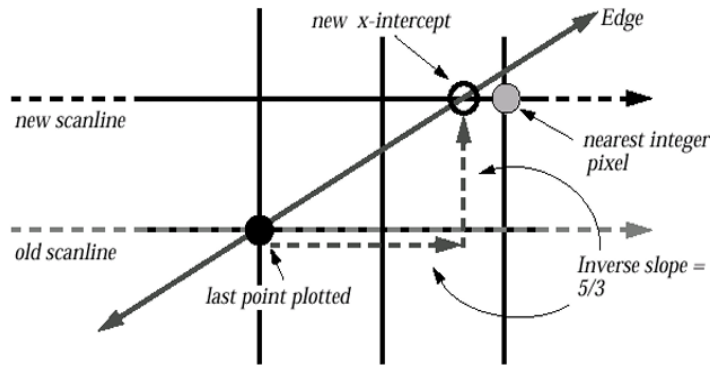
فتصبح المجموعتين بالشكل:

$$ET = \{\}, \quad AET = \{\}$$

ونكون هكذا وصلنا إلى نقطة نهاية تحريك الـ Scan Line، لاحظ أن ET، AET فارغتين.

الآن بعد ان قمنا بتحديد الأضلاع التي تتقاطع مع ال scan line في كل لحظة، كيف نحدد نقاط التقاطع ؟

نحن نعلم أن أول نقطة تقاطع مع أي ضلع هي بداية الضلع ونعلم أنه بكل حركة للأعلى لل scan line تزداد y بمقدار 1 ونعلم أيضاً أن نقاط التقاطع بين الضلع وال scan line تحقق معادلة هذا الضلع وبما أن أول نقطة تقاطع هي أخفض رأس في المضلع إذاً لحساب نقاط التقاطع الجديدة علينا إيجاد x وذلك بتعويض $y + 1$ بمعادلة المستقيم. فنستنتج أن ال x -intercept تزداد بمقدار $1/m$ (inverse slope) في كل مرة



$$y = mx + b$$

$$m = \frac{\Delta y}{\Delta x}$$

$$1) y = mx_{old} + b$$

$$2) y + 1 = mx_{new} + b$$

بتعويض 1 في 2 :

$$mx_{old} + b + 1 = mx_{new} + b$$

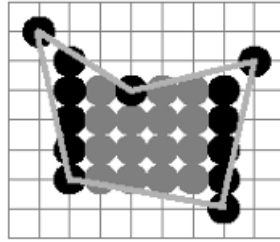
$$x_{new} = x_{old} + \frac{\Delta y}{\Delta x} = x_{old} + \frac{1}{m}$$

$$x_{new} = x_{old} + \frac{1}{m}$$

لدينا هنا مشكلة وهي تقريب $\frac{1}{m}$ لتجنب الأعداد الكسرية في الخوارزمية: نطبق خوارزمية Incremental بما أن m ثابت يمكن التقريب عن طريق جمع البسط حتى يصبح أكبر من المقام، أي يصبح الكسر أكبر من الواحد ثم نطرح المقام من البسط لنحسب كم تبقى فوق الواحد لنقوم بتخزينه : ثم نضيف لل x -intercept واحد .

بما أننا نملك نقطة من المضلع و ميل المضلع فيمكننا إيجاد النقطة التالية من المضلع باستخدام خوارزمية midpoint وهي بالتأكيد نقطة التقاطع التالية
 لاحظ أن حتماً جميع نقاط التقاطع هي من أضلاع الشكل قطعاً
 لكن إذا قمنا باستخدام خوارزمية midpoint كما في المحاضرة السابقة فسيقوم بأخذ البسكل الأقرب لكنه هكذا سيضيف إلى الشكل بكسلات غير موجودة فيه أساساً الشكل الذي نريد الحصول عليه :

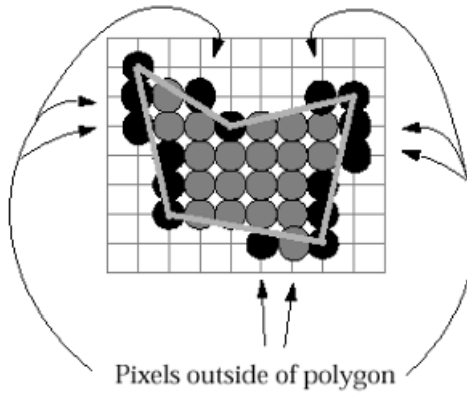
Extrema interior to polygon



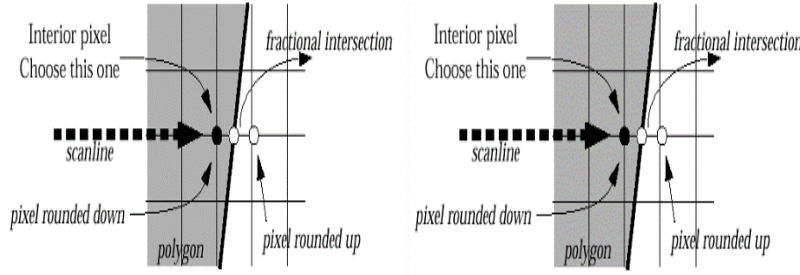
- Span extrema
- Other pixels in span

الشكل الذي سنحصل عليه باستخدام خوارزمية midpoint :

Extrema computed by midpoint algorithm



لذلك استخدمنا التقريب (rounding) لضمان ان جميع ال pixel تقع حتما داخل المضلع
 اذا كنت خارج المضلع و تقترب الى احد الاضلع الفردية نقوم بالتقريب للاعلى (Round Up) أي اذا كنت عند النقطة $x=5.4$ اختار 6 لضمان من دخول منطقة المضلع و اذا كنت داخل المضلع و تقترب من الحافة الزوجية فنقوم بالتقريب للاسفل (Round Down) لضمان عدم تلوين بكسل خارج منطقة المضلع فمثلا اذا كنت عند النقطة $x=10.7$ اختار 10
 أي إذا كانت الحافة اليسرى (حافة فردية) هي المختارة فإننا نختار ال pixel الواقع على اليمين، أما إذا كانت الحافة اليمنى (حافة زوجية) هي المختارة فنختار ال pixel الواقع على اليسار.



- فتكون الخوارزمية الأفضل هي خوارزمية Incremental، حيث كانت تحتاج إلى تقريب وهذا ما نريده في هذه الحالة

لكن في خوارزمية ال scan line في كل خطوة نزيد ال y بمقدار واحد وليس x ، لذلك سنزيد x بمقدار $\frac{1}{m}$ (لهذا السبب خزننا قيمة $\frac{1}{m}$ لكل ضلع). حيث m هي ميل الضلع وبما أننا نملك نقطة البداية فنستطيع إيجاد جميع نقاط التقاطع.

- بدلاً من استخدام الدالة round (التي يقرب إلى أقرب عدد صحيح)، إذا كانت الحافة يسارية فإننا نقوم بالتقريب إلى اليمين CEIL، أما إذا كانت الحافة يمينية فنقوم بالتقريب إلى اليسار باستخدام الدالة floor.

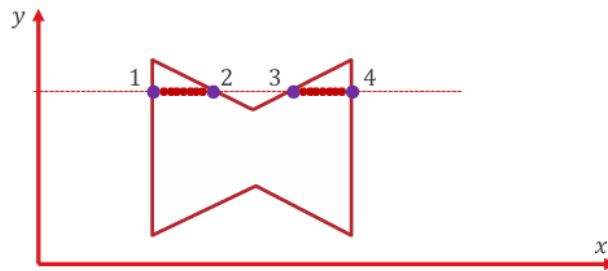
- **تذكرة:**

بفرض لدينا مستقيم ميله $m = \frac{\Delta y}{\Delta x}$ ، فإذا كانت النقطة $a(x_a, y_a)$ نقطة من ذلك المستقيم، فإن النقطة التالية إذا ضفنا قيمة واحد على ال x هي $b(x_{a+1}, y_{a+m})$ و النقطة التالية ل a إذا أضفنا قيمة واحد على y هي $c(x_{a+1/m}, y_{a+1})$

- في كل مستوى نأخذ جميع نقاط التقاطع ونرتبهم حسب قيم ال x من الأصغر إلى الأكبر، تكون البكسلات الموجودة ضمن هي البكسلات المحصورة بين نقاط التقاطع الفردية والزوجية.

ملاحظة ارييسيزية:

لو أن المضلع غير مركب (بسيط لا يتقاطع مع نفسه) فإننا نحتاج لترتيب نقاط التقاطع مرة واحدة، وبذلك يمكن تحسين الخوارزمية وتسريعها، وهذا ما تفعله OpenGL لكن عندما يكون المضلع مركباً فلا يمكننا ضمان بقاء نقاط التقاطع مرتبة دائماً، لذا نلجأ إلى إعادة ترتيبها في كل مستوى.



لاحظ أن نقطة التقاطع الأولى تمثل الدخول إلى الشكل، ونقطة التقاطع الثانية تمثل الخروج من الشكل. وبناءً على ذلك، فإن البكسلات الواقعة داخل الشكل هي البكسلات المحصورة بين نقاط التقاطع الفردية (التي تمثل الدخول إلى الشكل) ونقاط التقاطع الزوجية (التي تمثل الخروج من الشكل).

حساب خوارزمية التقاطع:

```

LeftEdgeScan (int xmin, int ymin, int xmax, int ymax, int value)
{
    int y;
    int x=xmin;
    int numerator=xmax-xmin;
    int denominator=ymax-ymin;
    int increment=denominator;
    for (y=ymin; y<=ymax; y++) {
        WritePixel(x,y,value)
        increment+=numerator;
        if (increment > denominator){
            /* Overflow, so round up to next pixel and decrement the
            increment. */
            x++;
            increment -=denominator;
        }
    }
} /*leftEdgeScan */

```

والآن، بعد أن أخذنا الفكرة العامة، دعونا ننتقل إلى بعض المشكلات التي نواجهها في هذه الخوارزمية وكيفية حلها.

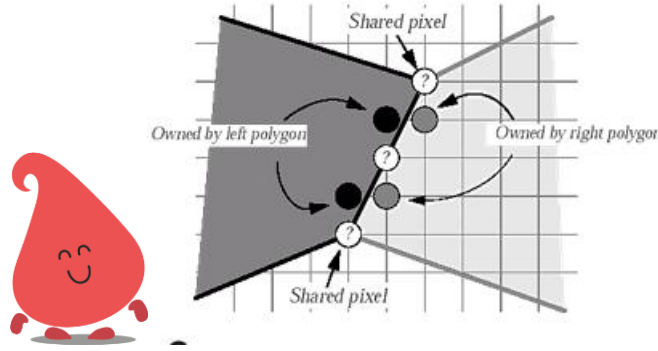
■ إن إحداثيات النقاط والقيمة $\frac{1}{m}$ هي قيمة float لكننا يجب ان نحدد موقع البكسل كقيمة integer بدلاً من تخزين قيمة float في كل مرة نقوم بعملية ceil / floor فستقوم بتخزين الجزء الصحيح على حدة في int ، والجزء العشري على حدة في float ، ونتعامل في تحديد البكسل مع الجزء الصحيح.

عند إجراء عملية الإضافة، نضيف القيمة الصحيحة إلى الجزء الصحيح، والقيمة العشرية إلى الجزء العشري. فإذا أصبح الجزء العشري أكبر من الواحد، نضيف الواحد إلى الجزء الصحيح ونطرح الواحد من الجزء العشري، وبذلك نحافظ على أن يبقى الجزء العشري أصغر من الواحد.

كما يمكننا أيضاً أن نمثل القيمة على شكل كسر مكون من بسط ومقام صحيحين بدلاً من التعامل مع الجزء العشري باستخدام float ، مما يسهل المعالجة ويزيد الدقة وعندما يصبح البسط أكبر من المقام، نضيف int من بسط ومقام صحيحين قيمة واحد إلى الجزء الصحيح ونطرح المقام من البسط، للحفاظ على أن يبقى الكسر أصغر من الواحد.

■ كما نعلم فإن البكسل الواحد ينتمي الى Polygon واحد, ماذا لو كان هنالك ضلع مشترك بين شكلين متجاورين.

ببساطة سنصطلح في مثل هذه الحالات على انه سنتنازل من اليمين عن البكسل لليسا, أي دائماً نأخذ ceil (دالة التقريب) للضلع اليساري و يتم استبعاد هذه البكسلات على الضلع اليميني, وهكذا سينزاح الشكل قليلاً نحو اليمين. بالتأكيد كان بإمكاننا اصطلاح العكس لكن هذه هي الطريقة المعتادة. لا يهمني من يأخذ البكسلات المهم ان لا يظهر وميض على الشاشة



ملاحظة 1:

أيضاً ال polygon يظهر فيها تكسر مثل المستقيمات و أيضاً أقوم بعمل Anti-Aliasing

ملاحظة 2 :

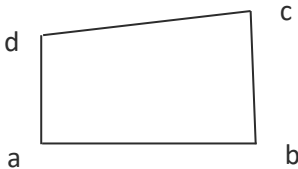
إذا كان لدينا ضلع عمودي فيكون جميع قيم x واحدة و بنفس فكرة الضلع المشترك يجب ان نحدد إذا كان هذا الضلع داخل او خارج ال polygon بنفس الخوارزمية

■ إذا كان لدينا ضلع أفقي , عندئذ ستكون y نقطة النهاية هي ذاتها y نقطة البداية , فمتى ستم إضافة إلى AET و متى سيتم حذفه, كيف ستم معالجته؟ والمشكلة الأكبر أن ميل الضلع الأفقي صفر, لذلك عند حساب مقلوب الميل فسنقع في Run Time Error فكيف نحل هذه المشكلة أيضاً؟

ببساطة فإننا سنتجاهل الأضلاع الأفقية من جدول الأضلاع ET .

لاحظ عند البداية سيتم إضافة الأضلاع bc , ad إلى AET و سنتجاهل ab ونحذفه نهائياً لأنه ضلع أفقي. A ستكون هي نقطة التقاطع الأولى و b هي نقطة التقاطع الثانية فسيتم ملء ما بينهما, ونكون بذلك تلقائياً رسمنا الضلع الأفقي ab رغم أننا أهملناه ولم نصفه إلى AET .

لذلك فإنه ليس هنالك داع لإضافة الأضلاع الأفقية إلى ET وبذلك نتجنب حساب المقدار $\frac{1}{m}$ لها.



سؤال امتحان: كيف نعالج مشكلة الضلع الافقي ؟ نحذفه من ET

Visible Surface Detection

بعد أن قمنا بتحديد كل شكل ماهي البكسلات التي تقابله في الشاشة لابد أن هذه الأشكال ستتقاطع وستحجب بعض الأشكال، فكيف سنحدد ماهي الأشكال التي ستظهر و تعطي قيمتها للبكسل؟

تدعى هذه العملية بـ Visible Surface Detection وهناك تصنيفين أساسيين :

1. Object space Method : تقوم بمقارنة الأشكال وتحديد أي الأوجه ستظهر، وذلك عند تعريف المشهد.

2. Image space Method : يتم المعالجة على مستوى البكسل لتحديد الشكل الذي سيعرض عليه، وهو الأكثر شيوعاً.

سنأخذ مثلاً على كل من التصنيفين:

1. Back-Face Detection:

كما نعلم عند تعريف الـ polygon يتم تعريف الناظم على سطحه أيضاً وفقاً لقاعدة اليد اليمنى (حسب ترتيب رسم النقاط)، فيتم تحديد الوجه الأمامي والوجه الخلفي بناءً على ذلك، فيمكننا باستخدام هذه الطريقة تحديد أي وجه سيتم رسمه وأي وجه سيتم إهماله لأنه إذا كانت الكاميرا تنظر إلى الوجه الخلفي فلا نقوم برسمه.

فإذا كانت الزاوية بين شعاع الـ Look at و ناظم الجسم أكبر من 90 وأصغر من 270 سنرى الجسم، وإلا فلن نراه، أو بصيغة أخرى إن كانت الزاوية بالقيمة المطلقة أصغر من 90 فلن نراه.

ونحن نعلم أن شعاع الـ look at بعد التحويل إلى مركز إحداثيات الكاميرا هو بالاتجاه السالب فإذا كانت قيمة الناظم موجبة (بعد النقل إلى مركز إحداثيات الكاميرا) سيظهر الجسم، وإن كان سالب فلن يظهر.

هذه العملية في الـ open GL تتم باستخدام التعليمة CullFace.

باستخدام هذه التقنية فنحن تقريباً وفرنا معالجة نصف وجوه المضلعات من اختبار الرؤية.

♦ لكن باستخدام هذه الطريقة إما أن يظهر الوجه كاملاً أو يختفي كاملاً، وبالتالي إذا كان لدينا وجه نريد إظهار جزء منه فقط فلا يمكننا استخدام هذه الطريقة.

تمرين: هام للامتحان

لدينا مثلث رؤوسه:

$A(2,1,-1)$

$B(5,3,-7)$

$C(-4,-1,-5)$

هذه النقاط في جملة إحداثيات الكاميرا هل يظهر على الشاشة ؟

يتحدد ذلك حسب جهة الناظم ونحصل على الناظم عن طريق جداء خارجي بين شعاعين.
إذاً نحتاج شعاعين:

لإيجاد الشعاع الأول نطرح إحداثيات النقطة الأولى من الثانية: $AB(3,2,-6)$
 لإيجاد الشعاع الثاني نطرح إحداثيات النقطة الثانية من الثالثة: $BC(-9,-4,2)$

ملاحظة: الطرح يكون بالترتيب لأننا نستخدم قاعدة اليد اليمنى أي ترتيب النقاط مهم.

حساب الناظم:

$$z = x_1 \times y_2 - x_2 \times y_1 = 3 \times (-4) - 2 \times (-9) \\ = -12 + 18 = 6 > 0$$

جهة الناظم نحو z الموجبة إذا المثلث يظهر

مثال اربيسزي:

$A(3,5,-2)$ $B(4,6,-1)$ $C(8,2,-3)$

$V_1(1,1,1)$ $V_2(4,-4,-2)$

$$x_1 \times y_2 - x_2 \times y_1 = 1 \times (-4) - (4) \times 1 \\ = -4 - 4 = -8 < 0$$

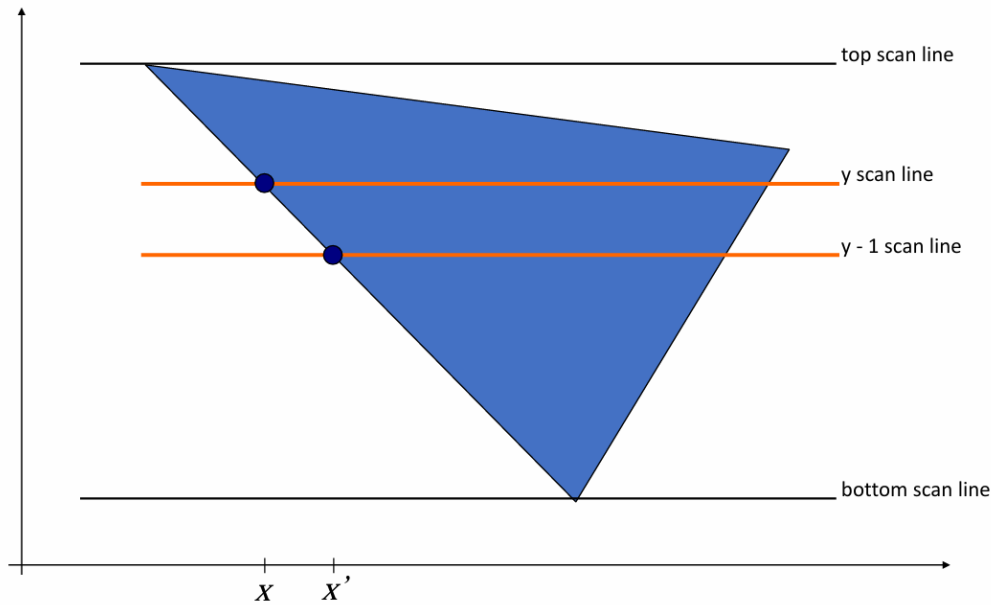
جهة الناظم نحو z السالبة إذا المثلث غير ظاهر.

2. Depth-BufferMethod (Z-Buffer) :

- من أجل كل بكسل نقوم بتخزين قيمة أقرب جسم إلى الكاميرا يقابل هذا البكسل.
- عندما نقوم بمعالجة شكل ما وملء البكسل نتأكد ان قيمة ال depth لهذه النقطة من الشكل أصغر من القيمة الحالية لهذا البكسل في ال depth buffer.
- إذا لا بد لنا من معرفة depth كل نقطة من ال polygon نحن نملك قيمة ال depth لرؤوس ال polygon قبل الإسقاط لذا يمكننا حفظها جانباً عند الإسقاط مع ال pixel.
- أما باقي نقاط الشكل فنحن سنستنتجهم من خلال تدرج العمق.

- نحن نعلم أن معادلة المستوي هي:
- $Ax + By + Cz + D = 0$ فيكون عمق أي نقطة منه هو:
- $z = (-Ax - By - D)/c$
- من أجل أي نقطة فإن مجاورتها على نفس السوية ستكون، وسيكون عمقها : $x' = x+1$
- $z' = (-A(x+1) - By - D)/C = -A/C + (-Ax - By - D)/C$
- الآن إن كان لدينا عمق فيمكننا حساب جميع النقاط الواقعة على نفس السوية.
- بالطريقة ذاتها إذا لو كان لدينا عمق نقطة من ضلع ميله m
- يمكننا حساب عمق النقطة التي أسفلها في ذلك الضلع باستخدام العلاقة:
- $x' = x-1/m$, $y' = y-1$
- $z' = z - (A/m + (B)/C) \leftarrow$
- إذا الآن أصبح بإمكاننا حساب عمق جميع النقاط في الشكل انطلاقاً من عمق رؤوس الشكل.

حسابات ال z غير مطلوبة



ثم من أجل كل بكسل نقوم بمقارنة عمق النقطة المقابلة مع قيمة ال Z-Buffer للبكسل، وإن كانت أصغر نقوم بتخزين قيمة النقطة اللونية في البكسل و تحديث قيمة ال Z-Buffer للبكسل.

سؤال هام للامتحان: احد الخطوات التي تطبق اثناء ال scan conversion : z-buffer shading

متى يكون ال z-buffer غير مفيد و نقوم بايقافه ؟ لرسم جبل على انه كبير جدا أقوم برسمه اولاً بالقرب من الشاشة ثم ارسم بعده الاجسام التي اريد فتظهر جميعها امامه
يوجد مشكلة لل z-buffer اذا كان الجسم القريب شفاف فهو لا يظهر الاجسام التي خلفه يفضل في هذه الحالة ان نرسم الاجسام من البعيد الى القريب

ملاحظات هامة:

- يوجد Buffer آخر يخزن القيمة اللونية للبكسل هو Frame Buffer أو يسمى أيضاً Color Buffer .
- القيمة الابتدائية ل Frame Buffer هو لون الخلفية المختار و افتراضياً هو الأسود.
- القيمة الابتدائية ل Z-Buffer هي واحد، لأن قيمة مستوي ال far plain clipping في ال canonical viewing volume هي واحد.

بعد أن حددنا كل بكسل إلى أي شكل ينتمي لم يتبقى لنا سوى تحديد القيمة اللونية لهذا البكسل و هذا ما سنتعرف عليه بالمحاضرة القادمة.

The END ...